

The C Programming Laboratory

A Working Guide to Vim, the C Compilation Model,
Variables, Operators and Basic Input/Output

Department of Computer Science and Engineering
Laboratory Manual Part II and Lab Assignment August 2026

(IMPORTANT**)** **How to use this manual.** Section 1 covers the vim editor. Section 2 is a reference to the concepts covered in class so far: what a compiler actually does, and what variables, constants and operators mean. **Read Section 2 before attempting any exercise.** Section 3 explains `printf()` and `scanf()`, and Section 4 contains the practice exercises. Every exercise can be solved using only the material in Sections 2 and 3.

1 Working with the Vim Editor in Linux

1.1 What is Vim?

vim (Vi IMproved) is a text editor available on almost every Linux system. It is used to write and edit program files directly from the terminal. Unlike Notepad, vim works in different **modes**, so it may feel different at first, but it becomes very fast once you get used to it.

1.2 Opening the Terminal

1. On your Linux system, open the **Terminal** application (search for “Terminal” or use the shortcut `Ctrl+Alt+T`).
2. Check your current location using:

```
pwd
```

3. (Optional) Create a folder for today’s lab and move into it:

```
mkdir lab1  
cd lab1
```

1.3 The Three Modes of Vim

Mode	Purpose	How to Enter
Normal Mode	Moving around, deleting, copying (default mode)	Press Esc
Insert Mode	Typing / editing text	Press i
Command Mode	Saving, quitting, searching	Press : (from Normal mode)

Tip for students: Vim always opens in **Normal Mode**. You must press `i` before you can start typing.

1.4 Step-by-Step: Create, Save, Compile and Run a C File

Step 1 Create a new file

```
vim first.c
```

This opens vim with a new empty file named `first.c`.

Step 2 Enter Insert Mode to type code. Press `i` on your keyboard. You should see `-- INSERT --` at the bottom of the screen.

Step 3 Type the following simple program

```
1 #include <stdio.h>
2
3 int main() {
4     printf("Hello World!\n");
5     return 0;
6 }
```

Step 4 Return to Normal Mode. Press `Esc` to exit Insert Mode.

Step 5 Save the file. In Normal Mode, type `:w` and press `Enter`. This saves (writes) the file without closing it.

Step 6 Save and quit together. Type `:wq`. This saves the file **and** exits vim, returning you to the terminal.

Step 7 Compile the C program. Back in the terminal, compile using `gcc`:

```
gcc first.c -o first
```

This creates an executable file named `first`.

Step 8 Run the program

```
./first
```

You should see `Hello World!` printed on the screen.

1.5 Other Common Vim Operations

All commands below are used in **Normal Mode** (press `Esc` first).

Task	Command
Move cursor left / down / up / right	Using arrow keys
Delete one character	x
Delete an entire line	dd
Delete a word	dw
Undo last action	u
Redo	Ctrl+r
Copy (yank) a line	yy
Paste copied/deleted text	p
Search for a word	/word then Enter
Go to next / previous match	n / N
Go to a specific line number	:25 (jumps to line 25)
Show line numbers	:set number
Replace one occurrence on a line	:s/old/new/
Replace every occurrence in the file	:%s/old/new/g
Compile without leaving the editor	:!gcc first.c -o first
Run without leaving the editor	!:./first
Quit without saving	:q!
Quit (only if no unsaved changes)	:q

Practice Exercise 1.1

1. Open a new file called `practice.c` using vim.
2. Type any 5 lines of text.
3. Delete line 3 using dd.
4. Undo the deletion using u.
5. Search for any word you typed using /.
6. Save and quit using :wq.

2 Understanding the C Language

2.1 Introduction to the C Language

C is a high level language originally developed on the UNIX operating system. It lets you write your programs clearly, and it has good control flow facilities so that your code can be read straight down the page. It lets you write structured code that is compact without being too cryptic; it encourages modularity and good program organisation; and it provides good data-structuring facilities.

High level language. A language written in a form close to human reasoning using words, names and algebraic expressions rather than in the numeric instructions the processor actually executes. A separate program, the *compiler*, performs the translation.

2.2 Creating the Program

- Create a file containing the complete program. You may use any ordinary editor you are familiar with; in this laboratory we use vim.
- The filename must, by convention, end in `.c` (a full stop followed by a lower case c) for example `firstprog.c`. The name before the full stop is chosen by you. The contents must obey C syntax.

Source code. The text of the program as you write it, in a `.c` file. It is readable by a human but cannot be executed by the machine until it has been compiled.

2.3 A First Program

```
1 #include <stdio.h>
2
3 void main()
4 {
5     printf("\nThis is my first program in C\n");
6 }
```

Save this code in the file `firstprog.c`. Now consider what each part of it means.

- A C program contains **functions** and **variables**. The functions specify the tasks to be performed by the program. The main function establishes the overall logic of the code; it is normally kept short and calls other functions to perform the necessary sub-tasks. **All C code must have a main function**, because execution always begins there.
- The program calls `printf`, an output function from the I/O (input/output) library, which is defined in the file `stdio.h`.
- The original C language did not have any built-in I/O statements whatsoever, nor much arithmetic functionality; it was not really intended for scientific or technical computation. These functions are now performed by **standard libraries**, which are part of ANSI C.
- The `printf` line prints the message `This is my first program in C` on *stdout*, the output stream corresponding to the terminal window in which you run the code. The `\n` prints a newline character, which brings the cursor onto the next line.
- **By construction**, `printf` **never inserts a newline on its own**. The following program therefore produces exactly the same result:

```
1 #include <stdio.h>
2
3 void main()
4 {
5     printf("\n");
6     printf("This is my first program in C");
7     printf("\n");
8 }
```

- The first statement, `#include <stdio.h>`, includes a specification of the C language I/O library.
- All variables in C must be explicitly defined before use. The `.h` files are by convention **header files**, containing definitions of variables and functions necessary for the functioning of a program, whether in code you have written yourself or as part of the standard C libraries.

- The directive `#include` tells the compiler to insert the contents of the specified file at that point in the C code. The `<...>` notation instructs the compiler to look for the file in certain standard system directories.
- The `void` preceding `main` indicates that `main` is of `void` type, that is, it has no type associated with it, meaning it cannot return a result on execution. (In modern practice `int main()` with `return 0;` is preferred, because it reports success to the operating system.)
- The `;` denotes the end of a statement.
- Blocks of statements are put in braces `{...}`, as in the definition of functions.
- All C statements are written in **free format**: there is no specified layout or column assignment.
- **Whitespace** (tabs or spaces) is never significant, *except* inside quotes as part of a character string.

2.4 Compilation

Compilation. The process of translating your source code into machine code that the processor can execute. The program that performs this translation is called a *compiler*.

- There are many C compilers. `cc` is the default Sun compiler; `gcc` is popular and available for many platforms, and is the one used in this laboratory.
- Save your C code in `firstprog.c`, then compile it by typing at the prompt:

```
cc firstprog.c          or          gcc firstprog.c
```

This creates an executable file called `a.out`, which is then executed simply by typing its name.

- If you want the executable stored in a file of your own choosing, use the `-o` option, which puts the compiled program into the file *anyname* instead of `a.out`:

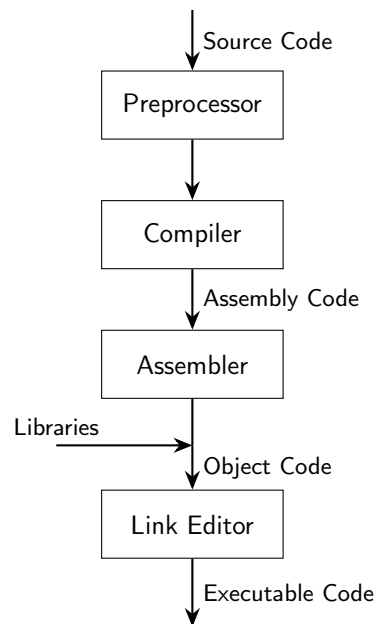
```
gcc -o anyname firstprog.c
./anyname
```

Two kinds of error

- **Syntax errors (compilation errors)** mistypings, misspelling a keyword, or omitting a semicolon. The compiler detects and reports these, and no executable is produced.
- **Logical errors** the compiler cannot detect these. Your program compiles and runs, but you have told the computer to do the wrong thing, such as dividing by zero or using the wrong formula. These appear only when you run the program and inspect the output. If so, you must return to edit your program source, recompile it, and run it again.

2.5 The C Compilation Model

Compilation is not one action but four, performed in sequence. Following are the key features of the C compilation model.



The Preprocessor

Preprocessor. A program that runs *before* the compiler. It accepts source code as input, removes all comments, and interprets special preprocessor directives, which are denoted by a hash #. Its output is still C code, but expanded.

The two directives you will meet first are:

- `#include` includes the contents of a named file, usually called a header file:

```
1 #include <stdio.h>      /* standard library I/O file */
2 #include <math.h>       /* standard library maths file */
```

- `#define` defines a symbolic name or constant. This is called *macro substitution*: every later occurrence of the name is replaced by the given text before compilation begins.

```
1 #define MAX_ARRAY_SIZE 100
```

The Compiler

Compiler. The stage that translates source code into *assembly code*, a low-level human-readable form specific to the processor. Its input is the output of the preprocessor.

The Assembler

Assembler. The stage that translates assembly code into *object code* actual machine instructions in binary. Object code cannot yet be run, because it may refer to library functions such as `printf` whose code is elsewhere.

The Link Editor

Link editor (linker). The stage that joins your object code with the object code of the libraries you used, resolving every such reference, and produces the final *executable code*.

2.6 C Program Structure

A C program basically has the following components:

- Preprocessor commands
- Type definitions
- Function prototypes these declare the type a function returns and the types of the variables passed to it
- Variables
- Function definitions
- The `main()` or `void main()` function. Some compilers do not accept `main()` on its own.
- C requires a semicolon at the end of every statement.

The overall structure of a C program is as follows:

```
1 main()
2 {
3     declarations      /* variables, arrays, records, function declarations,
4                        each one separated by a semicolon */
5
6     statements        /* each separated by a semicolon */
7 }
8
9 /* Function definitions */
```

For example:

```
1 #include <stdio.h>
2 #define i 6
3
4 void main()
5 {
6     int x, y;
7     x = 7;
8     y = i + x;
9     printf("%d\n", y);
10 }
```

A function has the form:

```
1 type function_name (parameters)
2 {
3     local variables
4     C statements
5 }
```

Note: if the type definition is omitted, C assumes the function returns an **integer** type. This can be a source of problems in a program.

2.7 Variables

Variable. A named location in the computer's memory that holds a value which may change while the program runs. Every variable has a *type*, which fixes how much memory it occupies and how the bits stored there are to be interpreted.

C has the following simple data types:

C type	Size (bytes)	Range
char	1	a single character
unsigned char	1	0 to 255
short int	2	-32768 to $+32767$
unsigned short int	2	0 to 65536
(long) int	4	-2^{31} to $+2^{31} - 1$
float	4	$\pm 3.2 \times 10^{38}$
double	8	$\pm 1.7 \times 10^{308}$

- On UNIX systems all ints are long ints unless specified as short int explicitly.
- **There is NO Boolean type in C.** Either use char, int, or better unsigned char. unsigned can be used with all char and int types.

Declaring a variable

To declare a variable in C, write the type followed by a list of names:

```
1 int    i, j, k;
2 float x, y, z;
3 char  ch;
```

Declaration. The statement that tells the compiler a variable's name and type, so that memory can be reserved for it. In C a variable *must* be declared before it is used.

C also allows **multiple assignment** statements using =, such as `a = b = c = d = 3;` This is the same as assigning to each variable separately but is more efficient. It is only possible if all the variables in the statement have the same type.

The preprocessor `#define` is another, more flexible method of defining constants in a program:

```
1 #define PI 3.14159
```

The difference is that `const` creates a real, typed variable that cannot be modified, whereas `#define` performs a textual substitution before compilation and creates no variable at all.

You will frequently see `const` in function parameters. This simply says that the function is not going to change the value of the parameter.

2.8 Escape Sequences the Backslash Characters

Escape sequence. A pair of characters beginning with a backslash, used inside a string to represent a character that cannot conveniently be typed. The backslash is not printed; it changes the meaning of the character that follows it.

Sequence	Meaning
\n	newline brings the cursor onto the next line
\t	horizontal tab useful for aligning columns of output
\\	prints a single backslash
\"	prints a double quote inside a string
\a	alert a beep from the terminal
\0	the null character, which marks the end of a string

2.9 Operators

Operator. A symbol that instructs the compiler to perform an operation on one or more values. The values it acts upon are called its *operands*, and the whole combination is an *expression*, which itself has a value.

Arithmetic operators

The standard arithmetic operators + - * / are found in most languages; C provides some more.

- **Assignment** is =, as in `i = 4;` or `ch = 'y';`
- **Increment** ++ and **decrement** -- are more efficient than their longhand equivalents: `x++` is faster than `x = x + 1.`
- The ++ and -- operators can be either **post-fixed** or **pre-fixed**. With pre-fixed the value is computed *before* the expression is evaluated, whereas with post-fixed the value is computed *after* the expression is evaluated.
- The % (**modulus**) operator gives the remainder and only works with integers.
- Division / is used for both integer and float division, so be careful. The answer to `x = 3 / 2` is 1 even if x is declared a float. If both arguments of / are integers, C performs integer division.

Comparison operators

- To test for equality use the **double** equality symbol ==, as in `if (x == y).`
- Beware of using = instead of ==, such as writing accidentally `if (i = j).` This is a perfectly **legal** C statement, syntactically speaking, which copies the value in j into i and delivers this value, which will then be interpreted as true if j is non-zero.
- Not equal to is !=. The operators < (less than), > (greater than), <= (less than or equal) and >= (greater than or equal) are as usual.

Logical operators

- The two basic logical operators are && for logical AND and || for logical OR. The unary operator ! is logical NOT and takes only one argument.
- The operators & and | have a *different* meaning: they are bitwise AND and OR.

Order of precedence

It is necessary to be careful of the meaning of an expression such as $a + b * c$. Do we want the effect of $(a + b) * c$ or of $a + (b * c)$?

All operators have a priority, and high priority operators are evaluated before lower priority ones. Operators of the same priority are evaluated from left to right, so that $a - b - c$ is evaluated as $(a - b) - c$. From high priority to low priority the order for all C operators is:

Highest	() [] -> .	
	! ~ & sizeof cast ++ -- + - (these are right->left)	
	* / %	
	+ -	
	< <= >= >	
	== !=	
	&	
	^	
	&&	
	?:	(right->left)
Lowest	= += -=	(right->left)
	, (comma)	

Thus $a < 10 \ \&\& \ 2 * b < c$ is interpreted as $(a < 10) \ \&\& \ ((2 * b) < c)$. When in doubt, use parentheses: they cost nothing and make your intention clear to anyone reading the program.

3 Printing Out and Inputting Variables

3.1 The printf() Function

C uses **formatted output**. The `printf` function has a special formatting character `%`; a character following this defines a certain format for a variable.

Syntax:

```
1 printf("text and format specifiers", variables);
```

Specifier	Used for
%d	Integer (whole number)
%f	Float (decimal number)
%c	Single character
%s	String (text)
%%	Prints a literal % character

The format statement is enclosed in `"..."` and the variables follow after it. **Make sure the order of the format specifiers and the variable data types match up**, since the compiler will not always warn you.

Controlling decimal places. Writing `%.2f` prints a float rounded to two decimal places, and `%8.2f` additionally pads it to a total width of eight characters.

Example

```
1 #include <stdio.h>
2 int main() {
3     int age = 18;
4     printf("My age is %d years\n", age);
5     return 0;
6 }
```

3.2 The scanf() Function

scanf() is the function for inputting values into a variable. Its format is similar to that of printf.

Syntax:

```
1 scanf("format specifier", &variable);
```

Important: the & before a variable means *address of* that variable. Variables appearing in scanf must always be preceded by & (except for strings and character arrays), because scanf needs to know *where* in memory to put the value it reads.

Example taking an integer input

```
1 #include <stdio.h>
2 int main() {
3     int rollNumber;
4     printf("Enter your roll number: ");
5     scanf("%d", &rollNumber);
6     printf("Your roll number is %d\n", rollNumber);
7     return 0;
8 }
```

Example taking a character input

```
1 #include <stdio.h>
2 int main() {
3     char grade;
4     printf("Enter your grade (A/B/C): ");
5     scanf(" %c", &grade); // note the space before %c
6     printf("Your grade is %c\n", grade);
7     return 0;
8 }
```

Tip: A space before %c is added to skip any leftover Enter key or newline from previous input.

Example taking multiple inputs together

```
1 #include <stdio.h>
2 int main() {
3     int day, month, year;
4     printf("Enter date as dd mm yyyy: ");
5     scanf("%d %d %d", &day, &month, &year);
6     printf("You entered: %d/%d/%d\n", day, month, year);
7 }
```

```
7   return 0;  
8 }
```

4 Practice Exercise 2.2

Note: you have not yet studied loops or `if` statements. Every question below can and should be solved using only variables, blocks, operators, `printf()` and `scanf()`.

1. Write a program using the **echo** command to print “Hello World”.
2. Search the word “Hello” in the program that you wrote in question 1 and replace it with “YOUR NAME” using vim commands.
3. Write a program that asks the user for their **name (as characters)**, **age (int)** and **height (float)**, then prints all three back using `printf()`.
4. Write a C program to find the **sum and average** of three numbers provided by the user. Read the three numbers with a single `scanf()`, and print the average correct to two decimal places.
5. Write a program that reads the **radius** of a circle as a `float` and prints its area and circumference. Declare π as a constant using either `const` or `#define`. Print both results to three decimal places.
6. Write a program that reads two integers `a` and `b` and prints, on three separate lines: their quotient using integer division, their remainder using `%`, and their quotient again as a decimal number. *Explain in a comment why the first and third answers differ.*
7. Write a program that reads a total number of seconds as an integer and converts it into **hours, minutes and seconds**. For example, an input of 7384 should print 2 h 3 m 4 s. Use only the `/` and `%` operators.
8. Write a program that reads a single character from the user and prints its **ASCII code**, then prints the character that comes immediately after it in the ASCII table. *Hint: a `char` may be cast to an `int`, and adding 1 to a character moves to the next one.*
9. Write a program that reads a single character from the user and prints the following pattern using that character. The pattern below is shown for the input `*`. Since loops have not yet been covered, use one `printf()` statement per line, and use `\t` or spaces for the alignment.

```
Enter a character: *
```

```
    *  
  * *  
* * *  
* * * *
```